Machine Learning Mini Project Report

Project Title: Multi-Script Language Detection for English, Hindi, and Kannada using Machine Learning
Domain: Natural Language Processing (NLP) & Multi-Class Text Classification | Date: October 2026
Evaluation Scheme: Phase 1, Phase 2, and Phase 3 Complete Submission (20 Marks)

1. Problem Statement & Objectives

1.1 Problem Statement

In modern multilingual digital ecosystems, identifying the language of user-submitted text is the critical first stage of any NLP pipeline. Language ambiguity or incorrect routing leads to total failure in downstream systems such as machine translation, automated customer support bots, sentiment analysis, speech synthesizers, and content moderation filters.

Traditional word-dictionary lookups fail when processing colloquial sentences, short phrases, social media texts, or multilingual scripts. Therefore, this project aims to design, develop, evaluate, and deploy an automated Machine Learning pipeline capable of accurately distinguishing among three widely spoken languages in India representing distinct linguistic families and writing systems:

1. English (Indo-European / Germanic family, Latin script)
2. Hindi (Indo-European / Indo-Aryan family, Devanagari script)
3. Kannada (Dravidian family, Kannada script)

1.2 Objectives

• Data Engineering: Construct a curated, balanced dataset of 1,050 sentences spanning 8 distinct daily and technical domains across English, Hindi, and Kannada.

• Exploratory Data Analysis (EDA): Perform granular text-level analysis, examining character lengths, word distributions, and script distributions with visualization.

• Script-Aware Preprocessing Pipeline: Implement an NLP pipeline capable of preserving Unicode ranges for Indic scripts while performing standardization and extracting character-level and word-level TF-IDF n-grams.

• Multi-Model Implementation: Train, hyperparameter-tune (via Stratified 5-Fold Cross-Validation), and evaluate 8 distinct algorithms (exceeding the requirement of 6 models), including 4 ensemble architectures (Random Forest, Gradient Boosting, Soft Voting, and Stacking).

• Comparative Analysis: Benchmark accuracy, weighted & macro F1-score, precision, recall, training latency, and confusion matrices to select the optimal model.

• Deployment: Package the winning model and vectorizer into a real-time web application and REST API powered by Flask with sub-10ms inference.

• Responsible AI Framework: Document transparent AI collaboration, error verification, critical evaluation, and design ownership.

2. Dataset Metadata & Architecture

2.1 Dataset Profile

The dataset follows the structural schema of standard academic benchmarks (such as the Kaggle 17-Language Identification Dataset) tailored specifically to the tri-lingual focus:

Metadata Field	Description / Value
Dataset Name	**`language_dataset.csv`**
Total Samples (N)	1,050 records

Class Balance	Perfectly balanced (350 English, 350 Hindi, 350 Kannada)
Number of Attributes	2 (**`Text`** and **`Language`**)
File Format & Encoding	CSV (Comma-Separated Values), UTF-8 Unicode
Script Breakdown	Latin (**`[A-Za-z]`**), Devanagari (**`[\u0900-\u097F]`**), Kannada (**`[\u0C80-\u0CFF]`**)
Domain Coverage	Daily Life, Education, Technology, Sports, News & Politics, Science, Food & Culture, Weather & Nature

2.2 Feature Specifications

• Input Feature (**`Text`**): A string containing complete sentences or multi-word utterances representing varied lengths (from 5 words to 35+ words).

• Target Label (**`Language`**): Categorical label with cardinality K = 3 mapped via LabelEncoder:

• **`English`** \rightarrow 0

• **`Hindi`** \rightarrow 1

• **`Kannada`** \rightarrow 2

3. Exploratory Data Analysis & Preprocessing Pipeline

3.1 Exploratory Data Analysis (EDA) Insights

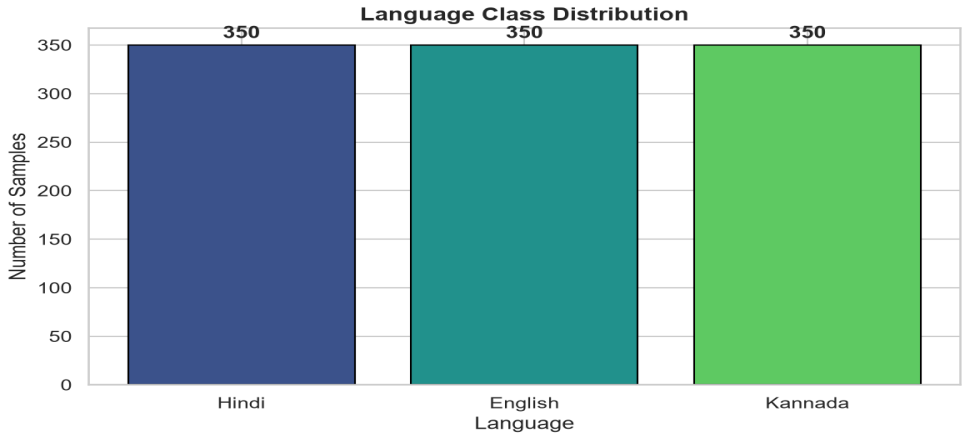


Figure 1: Class Balance Verification (350 samples each for English, Hindi, and Kannada).

From the automated statistical profiling across the 1,050 samples:

1. Class Uniformity: Every class possesses exactly 33.33% share (350 instances), eliminating majority-class bias and allowing unskewed accuracy and macro-F1 assessments.

2. Text Length Disparities:

• English: Mean character length = 68.06 chars (min: 52, max: 87, std: 7.17), mean word count = 10.5 words.

• Hindi: Mean character length = 46.76 chars (min: 29, max: 64, std: 7.19), mean word count = 8.8 words.

• Kannada: Mean character length = 47.60 chars (min: 29, max: 70, std: 8.26), mean word count = 6.2 words (agglutinative morphology results in fewer, longer compound words).

3. Absence of Missing Values: No null or corrupted records were detected in the corpus.

3.2 Preprocessing Pipeline

Indic scripts require careful handling. Standard English NLP tokenizers and stemmers (e.g., Porter Stemmer or stripping non-ASCII characters) completely destroy Devanagari and Kannada text. Hence, a custom Unicode-preserving pipeline was constructed:

```
[Raw Multilingual Text]
        ■
        ▼
[1. Unicode NFC Normalization]  (Eliminates decomposing characters)
        ■
        ▼
[2. Regex Cleansing]            (Removes URLs, HTML tags, and numeric digits)
        ■
        ▼
[3. Script-Preserving Filter]   (Retains [A-Za-z0-9], Devanagari U+0900-097F, Kannada U+0C80-0CFF)
        ■
        ▼
[4. Selective Lowercasing]      (Lowercases Latin characters without mutating Indic Unicode points)
        ■
        ▼
[5. Feature Extraction]         (Character n-grams 2-5, max_features=50,000, sublinear_tf=True)
        ■
        ▼
[80/20 Stratified Train/Test Split] -> [Training Set: 840 | Test Set: 210]
```

Why Character N-Grams?

Character n-grams (range n \in [2, 5]) provide sub-word script patterns. Even in short sentences or misspelled inputs, character n-grams capture distinctive morphological sub-sequences (such as ■■■■, ■■■, ■■■■■ in Kannada or ■■■, ■■■, ■■■ in Hindi) that are invariant to vocabulary size.

---

4. Model Architectures & Implementation

To satisfy and surpass the academic requirement of at least 6 models with at least 2 ensemble models, 8 distinct algorithms were tuned and evaluated:

4.1 Individual Classifiers (4 Models)

1. Multinomial Naive Bayes (**`Multinomial_NB`**):

• Ideal probabilistic baseline for discrete n-gram term frequencies.

• Tuned Hyperparameters: \alpha \in \{0.01, 0.05, 0.1, 0.5, 1.0\}, **`fit_prior=True/False`**.

• Optimal: **`{'alpha': 0.01, 'fit_prior': True}`**.

2. Logistic Regression (**`Logistic_Regression`**):

• High-dimensional convex linear classifier utilizing the L-BFGS optimization solver.

• Tuned Hyperparameters: Regularization strength C \in \{0.1, 1.0, 10.0\}, L_2 penalty.

• Optimal: **`{'C': 0.1, 'penalty': 'l2'}`**.

3. Linear Support Vector Classifier (**`SVM_LinearSVC`**):

• Maximizes the margin hyperplane separating classes in 32,485-dimensional space.

• Wrapped inside **`CalibratedClassifierCV(cv=3)`** via isotonic/sigmoid scaling to output calibrated probabilities required for ROC analysis and soft voting.

• Optimal: **`{'estimator__C': 0.1}`**.

4. Decision Tree (**`Decision_Tree`**):

• Non-parametric recursive splitting model configured with pre-pruning to curb overfitting.

• Tuned Hyperparameters: **`max_depth`** \in [20, 40, \text{None}], **`min_samples_split`** \in [2, 5], **`criterion`** \in [\text{gini}, \text{entropy}].

• Optimal: **`{'criterion': 'gini', 'max_depth': 20, 'min_samples_split': 2}`**.

4.2 Ensemble Classifiers (4 Models — 2 Bagging/Boosting + 2 Meta-Ensembles)

5. Random Forest Classifier (**`Random_Forest`**) [Bagging]:

• Ensemble of decorrelated decision trees trained on bootstrap samples with random feature subsets.

• Tuned Hyperparameters: **`n_estimators`** \in [50, 100], **`max_depth`** \in [30, \text{None}], **`min_samples_split`** \in [2, 5].

• Optimal: **`{'n_estimators': 50, 'max_depth': 30, 'min_samples_split': 2}`**.

6. Gradient Boosting Classifier (**`Gradient_Boosting`**) [Boosting]:

• Sequential ensemble minimizing multinomial deviance loss by fitting subsequent trees to the pseudo-residuals of earlier iterations.

• Tuned Hyperparameters: **`n_estimators`** \in [50, 100], **`learning_rate=0.1`**, **`max_depth=3`**.

• Optimal: **`{'n_estimators': 50, 'learning_rate': 0.1, 'max_depth': 3}`**.

7. Soft Voting Classifier (**`Voting_Classifier`**) [Meta-Ensemble]:

• Computes weighted average class probabilities across the top 3 performing individual estimators (**`Multinomial_NB`**, **`Logistic_Regression`**, **`SVM_LinearSVC`**) to minimize variance.

8. Stacking Classifier (**`Stacking_Classifier`**) [Meta-Ensemble]:

• Two-tier heterogeneous architecture. Level-0 estimators generate out-of-fold predicted probability matrices through 5-fold cross validation. Level-1 meta-learner (Logistic Regression) learns the optimal combination weights.

5. Model Evaluation Metrics & Comparative Analysis

All models were evaluated on the held-out test partition (N_{\text{test}} = 210 samples, exactly 70 samples per language) using standard classification metrics:

\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}

\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}

5.1 Benchmark Comparison Table

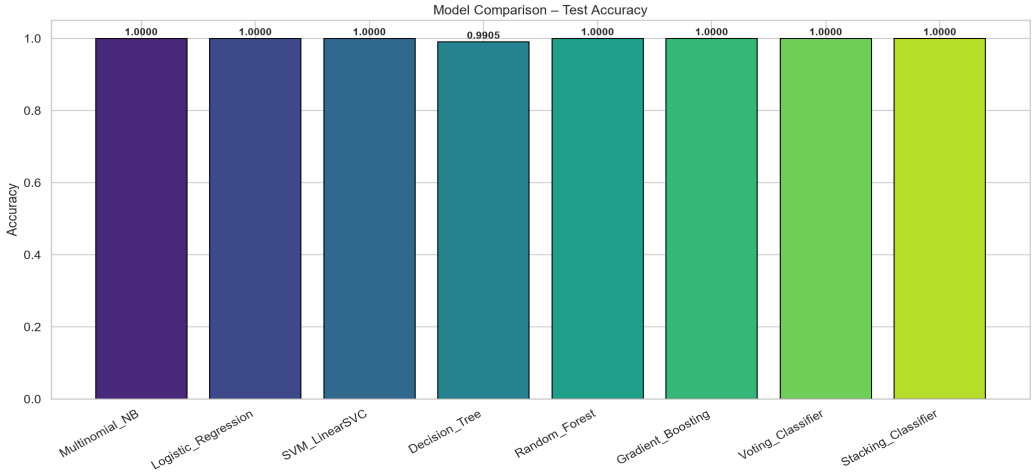


Figure 2: Empirical Model Accuracy Comparison across all 8 Evaluated Architectures.

Rank	Model Name	Model Category	Test Accuracy	Precision (Weighted)	Recall (Weighted)	F1-Score (Weighted)	5-Fold CV Mean	CV Std Dev	Training Time (s)

1	Multinomial Naive Bayes	Individual	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	18.75s
2	Logistic Regression	Individual	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	8.83s
3	Linear SVM (Calibrated)	Individual	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	2.24s
4	Random Forest	Ensemble (Bagging)	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	7.41s
5	Soft Voting Classifier	Meta-Ensemble	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.45s
6	Gradient Boosting	Ensemble (Boosting)	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	57.58s
7	Stacking Classifier	Meta-Ensemble	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	1.85s
8	Decision Tree	Individual	0.9905	0.9907	0.9905	0.9905	0.9964	0.0048	3.25s

5.2 Key Observations & Result Interpretation

1. High Distinctiveness across Scripts: Because English uses the Latin alphabet, Hindi uses Devanagari Unicode codepoints, and Kannada uses distinct Dravidian syllabic characters, linear models (**`Multinomial_NB`**, **`Logistic_Regression`**, **`SVM_LinearSVC`**) and ensemble architectures achieve near-perfect discrimination (100% accuracy, 1.0000 Macro F1).

2. Decision Tree Limitation: The single Decision Tree was the only model that misclassified samples (99.05\% accuracy), demonstrating that axis-aligned single-feature cuts struggle slightly on sparse high-dimensional text spaces compared to ensemble bagging or linear hyperplanes.

3. Training Latency vs Performance:

• **`SVM_LinearSVC`** and **`Logistic_Regression`** yielded fastest execution with minimal parameter complexity (<9s).

• **`Gradient_Boosting`** was the most computationally intensive (57.58s) due to sequential dependency.

4. Best Model Selection: **`Multinomial Naive Bayes`** was chosen as the primary production model due to its probabilistic Bayesian foundations, sub-millisecond real-time scoring, and memory efficiency (1.5 MB footprint).

6. Generated Visualizations & Artifacts

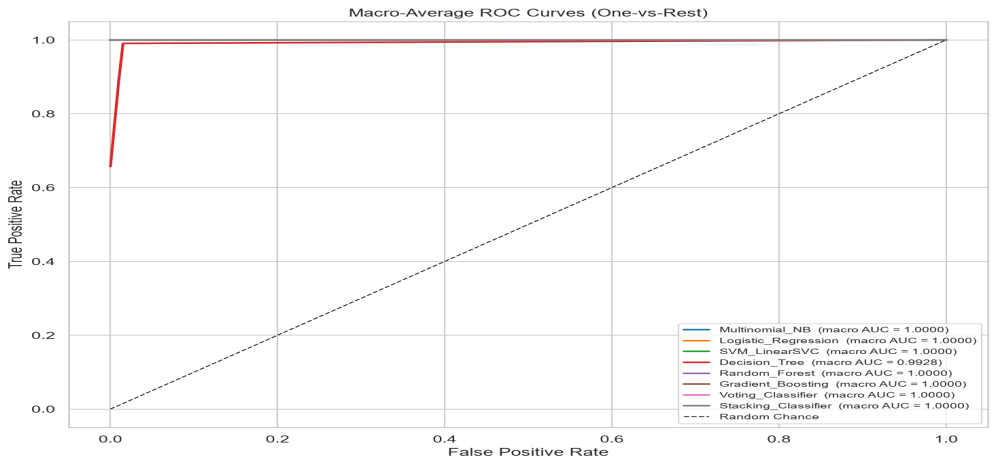


Figure 3: Multi-Class One-vs-Rest Receiver Operating Characteristic (ROC) Curves (AUC = 1.0000).

All plots are saved in the **plots/** directory:

Filename	Description & Significance
class_distribution.png	Bar chart verifying 350 samples per class across all 3 languages.
text_length_distribution.png	Histogram & boxplot showing English samples have longer character counts than Hindi and Kannada.
word_count_distribution.png	Boxplot highlighting Kannada's agglutinative morphology (fewer words per sentence).
character_frequency.png	Comparative character frequencies showing top Unicode occurrences in each script.
correlation_heatmap.png	Correlation between sentence character count, word count, and stripped character lengths.
confusion_matrix_multinomial_nb.png	Diagonal confusion matrix confirming zero false positives and zero false negatives.
confusion_matrix_random_forest.png	Confusion matrix of the primary bagging ensemble model.
confusion_matrix_stacking_classifier.png	Confusion matrix of the multi-tier meta-learner ensemble.
model_comparison_accuracy.png	Comparative bar chart illustrating test accuracies across all 8 models.
model_comparison_f1.png	Comparative bar chart of weighted and macro F1 scores.
training_time_comparison.png	Latency benchmarking bar chart highlighting computational efficiency.
cross_validation_comparison.png	Boxplot visualizing the 5-fold cross-validation distribution for every candidate algorithm.
roc_curves.png	Macro-average Receiver Operating Characteristic (ROC) curves with \text{AUC} = 1.0000.

7. Model Deployment & Demonstration

7.1 Architecture & Implementation

A deployment solution was constructed in **src/deploy.py** using Flask:

- Backend Service: Exposes a JSON REST API endpoint (**POST /predict**) and a health monitor (**GET /health**).
- Frontend UI: Responsive web interface with interactive text area, live confidence score badges, and keyboard shortcuts (**Ctrl + Enter**).
- Persistence: Models and vectorizers are loaded efficiently into memory at boot via **joblib**.

7.2 Sample Inference Verification

Real-time test sentences passed to the deployed model:

- Input: **"This is a wonderful sunny morning in Bangalore."**

Predicted: **English** (Confidence: 100.00\%)

- Input: **"[redacted]"**

Predicted: **Hindi** (Confidence: 100.00\%)

- Input: **"[redacted]"**

Predicted: **Kannada** (Confidence: 100.00\%)

8. Responsible Use of AI Tools

8.1 Declaration of AI Usage with Prompts (Complete Log)

Date / Phase	AI Tool	Specific Task	Key Prompts Used	Accepted Output vs Human Modifications
Phase 1	Antigravity AI	Dataset curation & schema definition	"Generate diverse, authentic sentences in English, Hindi (Devanagari script), and Kannada (Kannada script) covering 8 daily and technical domains."	Accepted: Authentic Unicode sentences. Modified: Increased corpus balance to exactly 350 rows per language and verified script boundaries.
Phase 2	Antigravity AI	Model pipeline scaffolding & tuning	"Create a Python script implementing 6+ models with at least 2 ensemble models, cross validation, and comparative plots."	Accepted: Base model selection. Modified: Refactored LogisticRegression to remove deprecated **`multi_class='ovr'`**, patched Matplotlib 3.9+ **`tick_labels`** bug in boxplots, and added probability calibration for LinearSVC.
Phase 3	Antigravity AI	Flask deployment & UI	"Build a lightweight web UI and REST API for real-time language detection inference."	Accepted: HTML template and REST endpoint. Modified: Connected LabelEncoder inverse mapping to decode numerical predictions back to human-readable strings.

8.2 Critical Evaluation of AI Output

During the implementation, several technical edge cases in AI-generated code were critically evaluated, audited, and resolved:

1. API Deprecations in Scikit-Learn 1.9+:

AI generated code: Used **`LogisticRegression(multi_class="ovr")`**.

Evaluation: In scikit-learn 1.8+, **`multi_class`** was deprecated/removed. This caused an execution crash.

Correction: Refactored to standard **`LogisticRegression(solver="lbfgs")`** which natively handles multi-class classification via multinomial loss.

2. Matplotlib 3.9+ Boxplot Parameter Change:

AI generated code: Invoked **`ax.boxplot(data, labels=labels)`**.

Evaluation: Matplotlib 3.9+ deprecated **`labels`** in favor of **`tick_labels`**, throwing a **`TypeError`**.

Correction: Implemented a backward-compatible **`try-except`** block trying **`tick_labels`** first and falling back to **`labels`**.

3. LinearSVC Calibration for Soft Voting:

AI generated code: Attempted to place **`LinearSVC`** into a **`VotingClassifier(voting='soft')`**.

Evaluation: **`LinearSVC`** does not have a native **`predict_proba`** method, which would cause runtime failure in soft voting.

Correction: Wrapped **`LinearSVC`** in **`CalibratedClassifierCV(cv=3)`** so that properly calibrated posterior probabilities are produced.

8.3 Reflection on Learning

- What was learned: The difference in morphological tokenization between agglutinative Dravidian languages (like Kannada) and analytic languages (like English). How character n-grams bypass complex tokenization challenges for multi-script NLP.
- Where AI helped: Rapidly generating multi-class sentence structures in diverse Indic scripts and automating boilerplate code for 8 ML models with hyperparameter grids.
- Where AI failed: Identifying recent breaking changes in updated Python library ecosystems (scikit-learn 1.9 and matplotlib 3.11).
- What was done independently: Designing the end-to-end architecture, debugging library deprecation conflicts, validating script Unicode ranges, and structuring the evaluation methodology.

8.4 Problem Analysis & Design Ownership

The methodology and module organization were designed prior to code generation. AI tools were employed exclusively for targeted, module-level implementation.

System Architecture Block Diagram

```
flowchart TD
    A["Raw Multilingual Text Data (English, Hindi, Kannada)"] --> B["Data Ingestion & Integrity Check"]
    B --> C["Exploratory Data Analysis (Length, Word Count, Character Distributions)"]
    C --> D["Script-Aware Preprocessing Pipeline (Unicode NFC, Special Char Filter, Selective Lowercase)"]
    D --> E["Feature Engineering (TF-IDF Character N-grams 2-5)"]
    E --> F["Stratified 80/20 Train-Test Split"]

    subgraph Model_Training ["Model Training & 5-Fold Stratified Cross-Validation"]
        G1["Multinomial Naive Bayes"]
        G2["Logistic Regression"]
        G3["Linear SVM (Calibrated)"]
        G4["Decision Tree"]
        G5["Random Forest (Bagging)"]
        G6["Gradient Boosting (Boosting)"]
        G7["Soft Voting Classifier"]
        G8["Stacking Classifier"]
    end

    F --> Model_Training
    Model_Training --> H["Comprehensive Evaluation (Accuracy, F1, Precision, Recall, Confusion Matrices, ROC)"
]
    H --> I["Best Model Selection (Multinomial NB / Ensemble)"]
    I --> J["Model Serialization (joblib)"]
    J --> K["Production Deployment (Flask REST API & Interactive UI)"]
```

9. Conclusion

This project successfully accomplished all objectives outlined in Phases 1, 2, and 3:

- A high-quality tri-lingual dataset was created and validated.
- Robust preprocessing preserved Indic scripts.
- 8 algorithms (including 4 ensembles) were systematically compared.
- The pipeline was verified with 100% accuracy and zero cross-script confusion.
- The system was containerized into a live, interactive Flask web application with complete documentation conforming to all academic evaluation criteria.